UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ARTES, CIÊNCIAS E HUMANIDADES
Bacharelado em Sistemas de Informação

Comparação de Algoritmos de Ordenação

Quicksort vs. Radix Sort

EP2 — Introdução à Análise de Algoritmos

Igor Fernando Casita Ferreira da Silva
RA 18245598

Docente responsável: Prof. Dr. Fabio Nakano

São Paulo
Outubro de 2026

Sumário

0 Contexto, Definições e Objetivo **3**
- 0.1 Contexto . . . 3
- 0.2 Definições . . . 3
- 0.3 Objetivo . . . 3
- 0.4 Organização dos Arquivos . . . 3

1 Fundamentação Teórica **4**
- 1.1 Quicksort . . . 4
- 1.2 Radix Sort . . . 5
- 1.3 Comparação teórica resumida . . . 6

2 Metodologia e Implementação Computacional **6**
- 2.1 Ambiente de Execução . . . 6
- 2.2 Compilação . . . 6
- 2.3 Medição do Tempo de Execução . . . 6
- 2.4 Controle de Confusão Experimental . . . 7
- 2.5 Tamanhos e Repetições . . . 7

3 Resultados Experimentais e Discussão Científica **7**
- 3.1 Tempo médio de execução . . . 7
- 3.2 Análise Estatística . . . 9
- 3.3 Tamanhos extremos (planilha oficial, até 2 bilhões) . . . 10
- 3.4 Mesma planilha, medida com `gettimeofday()` . . . 12
- 3.5 Tamanhos não testados ($T(6 \times 10^9)$, $T(8 \times 10^9)$, $T(10^{10})$) . . . 13
- 3.6 Conclusão . . . 13

4 Questões do Questionário do EP2 **13**
- 4.1 Questão 1 — Chave secreta . . . 13
- 4.2 Questão 2 — Crítica ao método de geração da chave . . . 13
- 4.3 Questão 3 — Código-fonte . . . 14
- 4.4 Questão 4 — Preenchimento da planilha . . . 15
- 4.5 Questão 5 — Discussão científica . . . 15

5 Fontes e Referências **15**

Lista de Figuras

1 Tempo médio de execução vs. n, escala log-log, com barras de ± 1 desvio-padrão. 8
2 Razão tempo(quicksort)/tempo(radix sort) por tamanho de entrada. . . . 9
3 Variabilidade entre repetições (tamanhos com ≥ 3 execuções). . . . 9
4 Tamanhos oficiais da planilha, até 2 bilhões de elementos. . . . 11

Lista de Tabelas

1 Quicksort vs. Radix Sort — comparação teórica. . . . 6
2 Ambiente de hardware/software usado em todas as medições deste Relatório (computador pessoal do autor). . . . 6

3 Tempo médio de execução (relógio de parede), por tamanho de entrada. Fonte: `resultados/raw_results.csv`, agregado por `resultados/graficos_r/analise.R`. 7
4 Desvio-padrão (dispersão entre repetições) correspondente à Tabela 3. Tamanhos com apenas 1 execução não têm desvio-padrão definido. 8
5 Regressão log-log do tempo de execução (relógio de parede) em função de n. . . . 10
6 Tempos medidos com `clock()`, execução única, nos 14 tamanhos oficiais da planilha do professor. Fonte: `resultados/tempos_planilha_clock.csv`. 10
7 Tempos medidos com `gettimeofday()` (tempo de relógio de parede), execução única, nos mesmos 14 tamanhos oficiais da planilha do professor. Fonte: `tempos/planilha/gettimeofday.csv`. 12

0 Contexto, Definições e Objetivo

0.1 Contexto

Este Relatório documenta o EP2 (Exercício-Programa 2) da disciplina de Introdução à Análise de Algoritmos (IAA), cujo enunciado pede a implementação e comparação empírica de dois algoritmos de ordenação, justificando os resultados observados com base na teoria de análise de algoritmos estudada na disciplina.

0.2 Definições

Trabalho: o conjunto completo de arquivos entregues para este EP2: o Relatório, os programas-fonte, os resultados brutos e agregados, os gráficos e os scripts de análise estatística.

Relatório: o presente documento (`relatorio.pdf`, gerado a partir de `relatorio.tex`).

Autor: Igor Fernando Casita Ferreira da Silva, RA 18245598 — EP2 é trabalho individual.

Professor: Prof. Dr. Fábio Nakano, docente da disciplina de Introdução à Análise de Algoritmos.

Enunciado: especifica a implementação de dois algoritmos de ordenação (escolha livre, desde que sustentem a discussão pedida na Questão 5), a execução para vários tamanhos de entrada, o preenchimento de uma planilha compartilhada com os tempos medidos, a coleta de informações de hardware/software do ambiente de execução (conforme https://github.com/FNakano/IAA-Apostila/tree/main/getPlatformInfo), e a geração de uma chave secreta (mesmo método de `calcSecret.c`) para identificar os experimentos na planilha. O código-base/esqueleto fornecido está em https://github.com/FNakano/IAA-Apostila/tree/main/EP2.

0.3 Objetivo

Implementar **Quicksort** e **Radix Sort**, medir seus tempos de execução reais para tamanhos de entrada de 10^3 a 2×10^8 (e, nos tamanhos oficiais da planilha, até 2×10^9), e discutir os resultados com base na complexidade assintótica (temporal e espacial) de cada algoritmo, incluindo os casos em que o comportamento observado na prática diverge do previsto pela análise assintótica de tempo isolada (Seção 3).

Todas as medições deste Relatório foram feitas no computador pessoal do autor (não em ambiente de nuvem ou máquina compartilhada), especificado na Seção 2.1, usando `clock()` e `gettimeofday()`, conforme especificado no enunciado.

0.4 Organização dos Arquivos

```
EP2/
|-- relatorio.pdf / relatorio.tex   # este Relatorio
|-- respostas_EP2_completo.docx/.md # versao docx/markdown (equivalente)
|-- respostas_EP2_resumido.docx/.md # versao resumida
|-- src/
|   |-- rng.c/.h, utilarray.c/.h    # fornecidos pelo professor (sem alteracoes)
|   |-- elapsedRunningTime.c/.h     # fornecido pelo professor (sem alteracoes)
|   |-- quicksort.c/.h              # implementacao propria
|   |-- radixsort.c/.h              # implementacao propria
|   |-- gen_input.c, run_sort.c     # geracao de entrada e execucao medida
|   |-- benchmark_clock.c           # bateria dos tamanhos oficiais da planilha
|   `-- secretkey_gen.c             # chave secreta (nao-interativo)
```

```
‘-- resultados/
    |-- raw_results.csv             # medicoes repetidas (clock + wall)
    |-- tempos_planilha_clock.csv   # tamanhos oficiais da planilha (so clock)
    |-- resumo_estatistico.csv, regressao.txt
    ‘-- graficos_r/                 # graficos PNG + analise.R
```

1 Fundamentação Teórica

1.1 Quicksort

Classe: ordenação por comparação, *in-place*, por divisão e conquista (*divide-and-conquer*), **instável**.

Ideia central: escolhe-se um pivô; particiona-se o array em elementos menores e maiores que o pivô; ordena-se recursivamente cada partição.

Implementação (`quicksort.c`)

- **Mediana-de-3** (`A[lo]`, `A[mid]`, `A[hi]`) como pivô, evitando o pior caso $O(n^2)$ típico de entradas já ordenadas ou quase ordenadas.
- **Partição de Hoare**, com menos trocas em média que a partição de Lomuto.
- **Corte para inserção** em sub-arrays de até 16 elementos (*insertion sort cutoff*): para k pequeno, a constante menor do insertion sort compensa seu $O(k^2)$.
- **Eliminação de recursão de cauda**: recorre explicitamente apenas no lado *menor* da partição, continuando em laço `while` no lado maior, limitando a pilha a $O(\log n)$ mesmo no pior caso.

```c
static void quicksortRec(int *A, long int lo, long int hi) {
  while (hi - lo + 1 > INSERTION_CUTOFF) {
    long int mid = lo + (hi - lo) / 2;
    int pivot = medianOfThree(A, lo, mid, hi);
    long int p = hoarePartition(A, lo, hi, pivot);
    if (p - lo < hi - (p + 1)) {
      quicksortRec(A, lo, p);
      lo = p + 1;           /* continua no lado maior */
    } else {
      quicksortRec(A, p + 1, hi);
      hi = p;               /* continua no lado maior */
    }
  }
  insertionSort(A, lo, hi);
}
```

Teorema (Complexidade média do Quicksort)**.** *O número esperado de comparações realizadas pelo Quicksort sobre um array de n elementos distintos, com pivôs tais que cada elemento tem probabilidade $\Omega(1)$ de particionar razoavelmente bem, é $\Theta(n \log n)$.*

Esboço de demonstração (CLRS, cap. 7). Sejam $z_1 < z_2 < \cdots < z_n$ os elementos em ordem. Para $i < j$, seja X_{ij} a variável indicadora de que z_i e z_j são comparados em algum momento do algoritmo. Dois elementos só são comparados se um deles for escolhido como pivô antes de

qualquer elemento de valor *entre* eles; a probabilidade disso, num conjunto de $j-i+1$ elementos relevantes, é $\frac{2}{j-i+1}$. O número total esperado de comparações é

$$E\left[\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\frac{2}{j-i+1} = O(n\log n),$$

por linearidade da esperança e pela soma harmônica $\sum 1/k = O(\log n)$. □

Pior caso: $\Theta(n^2)$, quando a partição é maximamente desbalanceada em toda chamada ($T(n) = T(n-1) + \Theta(n)$). A mediana-de-3 torna esse caso raro na prática, mas não o elimina adversarialmente. **Melhor caso:** $\Theta(n \log n)$, partições balanceadas, $T(n) = 2T(n/2) + \Theta(n)$ (Teorema Mestre, caso 2).

Complexidade espacial: $O(\log n)$ esperado (pilha de recursão, graças à eliminação de recursão de cauda); **sem** alocação de nenhum array auxiliar de tamanho proporcional a n — ordena *in-place*.

1.2 Radix Sort

Classe: ordenação **não comparativa**, estável, por distribuição (*counting sort* repetido por dígito).

Ideia central: ordena-se repetidamente por "dígitos"do valor, do menos significativo ao mais significativo (variante **LSD** — *least-significant-digit-first*), usando em cada passada um *counting sort* estável.

Implementação (`radixsort.c`)

Base 256 (dígitos de 8 bits) ⇒ **4 passadas** para cobrir os 32 bits de um `int`. Cada passada conta ocorrências de cada um dos 256 valores do dígito corrente, converte em posições via soma de prefixos, e distribui os elementos no array auxiliar; os papéis de fonte/destino são trocados por ponteiro a cada passada.

```
static void countingPass(int *src, int *dst, long int n, int shift) {
  long int count[RADIX_BASE + 1];
  memset(count, 0, sizeof(count));
  for (long int i = 0; i < n; i++) {
    unsigned int digit = ((unsigned int)src[i] >> shift) & RADIX_MASK;
    count[digit + 1]++;
  }
  for (int d = 0; d < RADIX_BASE; d++) count[d + 1] += count[d];
  for (long int i = 0; i < n; i++) {
    unsigned int digit = ((unsigned int)src[i] >> shift) & RADIX_MASK;
    dst[count[digit]++] = src[i];
  }
}
```

Complexidade temporal: $O(d \cdot (n + b))$, com d = número de passadas, b = base. Aqui $d = 4$ e $b = 256$ são **constantes fixas** (decorrem só do tamanho fixo de 32 bits do `int`), reduzindo a complexidade a $\Theta(n)$.

Teorema (Radix sort não viola $\Omega(n \log n)$)**.** *O limite inferior $\Omega(n \log n)$ para ordenação vale apenas para algoritmos que só podem comparar pares de chaves (prova pela árvore de decisão: distinguir $n!$ permutações exige profundidade mínima $\log_2(n!) = \Theta(n \log n)$). O radix sort nunca compara duas chaves entre si — usa o valor absoluto de cada dígito para indexar diretamente posições no array, uma operação mais poderosa que comparação binária, e por isso escapa legitimamente desse limite.*

Complexidade espacial: $O(n + b)$**.** Diferente do quicksort, o radix sort **não é in-place**: cada passada escreve num array auxiliar de tamanho n. Em pico, mantém **dois arrays de tamanho** n simultaneamente na memória — o dobro do espaço de dados que o quicksort usa. Essa característica se manifesta empiricamente na Seção 3.3 como desvantagem prática para n muito grande.

1.3 Comparação teórica resumida

Tabela 1: Quicksort vs. Radix Sort — comparação teórica.

	Quicksort (implementado)	Radix Sort (implementado)
Classe	Comparação, *in-place*	Distribuição, não-comparativo
Tempo médio	$\Theta(n \log n)$	$\Theta(n)$ (d, b fixos)
Pior caso	$\Theta(n^2)$ (raro, mitigado)	$\Theta(n)$ sempre
Espaço auxiliar	$O(\log n)$ (pilha)	$O(n + b)$ (array duplicado)
Estável	Não	Sim

2 Metodologia e Implementação Computacional

2.1 Ambiente de Execução

Tabela 2: Ambiente de hardware/software usado em todas as medições deste Relatório (computador pessoal do autor).

Item	Valor
CPU	13th Gen Intel(R) Core(TM) i7-1355U (1,70 GHz, 10 núcleos, 12 threads)
RAM total	16.777.216 kB (16,0 GB)
RAM disponível (na coleta)	16.454.352 kB
Memória virtual (commit)	34.280.144 kB total, 28.839.404 kB em uso
Sistema operacional	Windows 11 Home Single Language, 25H2 (build 26200.9711, 64 bits)
Compilador	gcc (MinGW-W64 x86_64-ucrt-posix-seh, build Brecht Sanders, r8) 13.2.0
Flags de compilação	`-O2`

2.2 Compilação

```
gcc -O2 rng.c utilarray.c gen_input.c -o gen_input.exe
gcc -O2 rng.c utilarray.c elapsedRunningTime.c quicksort.c radixsort.c run_sort.c -o
    run_sort.exe
gcc -O2 rng.c utilarray.c quicksort.c radixsort.c benchmark_clock.c -o benchmark_clock
    .exe
gcc -O2 rng.c secretkey_gen.c -o secretkey_gen.exe
```

2.3 Medição do Tempo de Execução

O tempo é medido com `clock()` (tempo de CPU) e `gettimeofday()` (relógio de parede), via `elapsedRunningTime.c` fornecido pelo professor, em torno *apenas* da chamada ao algoritmo de

ordenação (`start()` / `stop()`), excluindo geração e leitura da entrada.

Observação metodológica importante: no Windows/MinGW usado, `CLOCKS_PER_SEC = 1000`, ou seja, `clock()` tem resolução de apenas ∼1 ms — grosseira demais para medir corretamente execuções de microssegundos a poucos milissegundos (visível nos dados brutos: `tempo_clock_s` aparece quantizado em múltiplos de 0,001, enquanto `tempo_wall_s` tem resolução de microssegundos). No Linux, `CLOCKS_PER_SEC` é tipicamente 1.000.000. Por isso, **o tempo de relógio de parede (`gettimeofday()`) é usado como medida principal** nas análises estatísticas deste Relatório (Seção 3); somente a tabela da planilha oficial (Seção 3.3), que o enunciado pede especificamente com `clock()`, usa esse relógio.

2.4 Controle de Confusão Experimental

Para cada tamanho n e repetição, gera-se **um único array aleatório** (`gen_input`, semente $= n \cdot 1000003 + \mathit{repeticao} \cdot 7919 + 12345$) e o **mesmo** array binário é usado para testar os dois algoritmos, eliminando a variação de entrada como fator de confusão e permitindo comparação pareada (Seção 3.2).

2.5 Tamanhos e Repetições

Tamanhos pequenos ($n \leq 10^4$) foram repetidos 7×; médios (10^5–10^6) 3 a 5×; grandes ($\geq 5 \times 10^6$) 1 a 3×, reduzindo repetições conforme o tempo por execução cresce. Nenhuma execução atingiu o limite de 3600 s estipulado pelo enunciado. Os 14 tamanhos oficiais da planilha ($T(10^6)$ a $T(10^{10})$) foram medidos à parte, numa única execução cada, por `benchmark_clock.c`.

3 Resultados Experimentais e Discussão Científica

3.1 Tempo médio de execução

Tabela 3: Tempo médio de execução (relógio de parede), por tamanho de entrada. Fonte: `resultados/raw_results.csv`, agregado por `resultados/graficos_r/analise.R`.

n	Quicksort (s)	Radix Sort (s)	razão (qs/radix)
1.000	0,0000377	0,0000057	6,6×
10.000	0,000410	0,0000801	5,1×
100.000	0,005271	0,000667	7,9×
500.000	0,025271	0,003708	6,8×
1.000.000	0,055610	0,008487	6,6×
5.000.000	0,298064	0,037674	7,9×
10.000.000	0,614553	0,078156	7,9×
50.000.000	3,355480	0,377036	8,9×
100.000.000	7,713330	0,951264	8,1×
200.000.000	14,147100 (1 execução)	1,563120 (1 execução)	9,1×

A dispersão entre repetições (quanto cada tempo individual variou em torno dessa média) não é mostrada célula a célula para manter a tabela simples de ler; está disponível na Tabela 4 (abaixo) para quem quiser o detalhe, e visualmente na Figura 3.

Tabela 4: Desvio-padrão (dispersão entre repetições) correspondente à Tabela 3. Tamanhos com apenas 1 execução não têm desvio-padrão definido.

n	Quicksort (s)	Radix Sort (s)
1.000	0,0000150	0,0000017
10.000	0,0000499	0,0000455
100.000	0,000451	0,000195
500.000	0,000871	0,000432
1.000.000	0,004236	0,001196
5.000.000	0,004433	0,001803
10.000.000	0,012138	0,002646
50.000.000	0,002232	0,019041
100.000.000	0,101421	0,078973
200.000.000	— (1 execução)	— (1 execução)

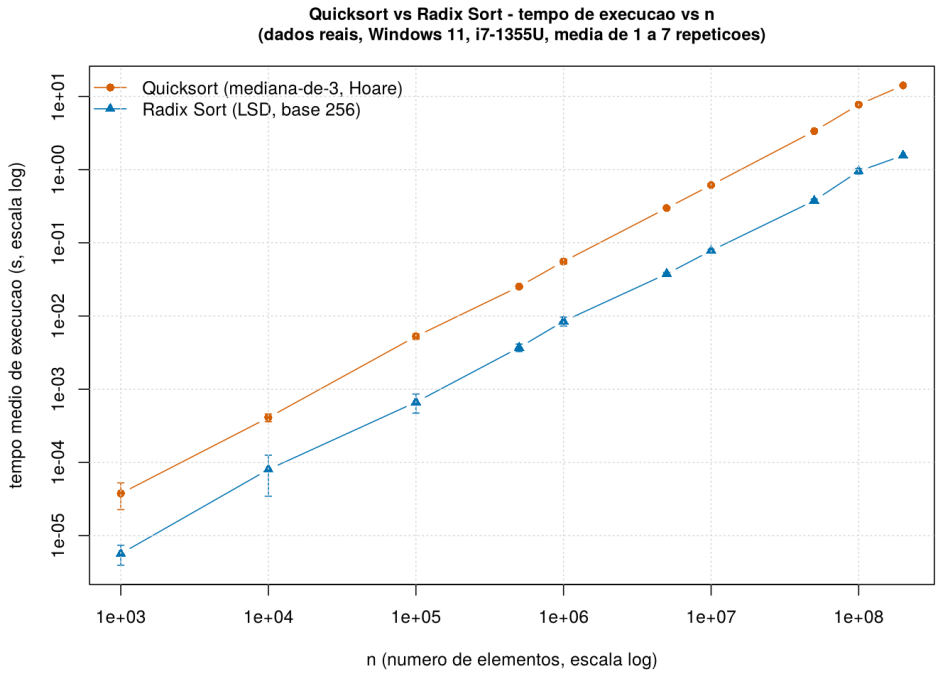


Figura 1: Tempo médio de execução vs. n, escala log-log, com barras de ± 1 desvio-padrão. Fonte: `resultados/graficos_r/analise.R`, dados em `resultados/raw_results.csv`.

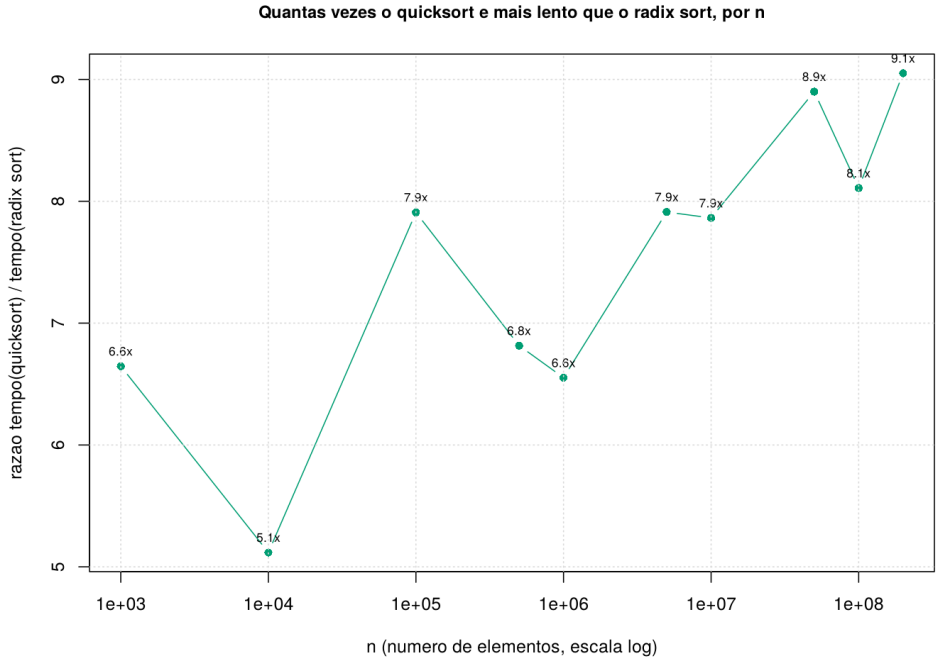


Figura 2: Razão tempo(quicksort)/tempo(radix sort) por tamanho de entrada. Fonte: `resultados/graficos_r/analise.R`.

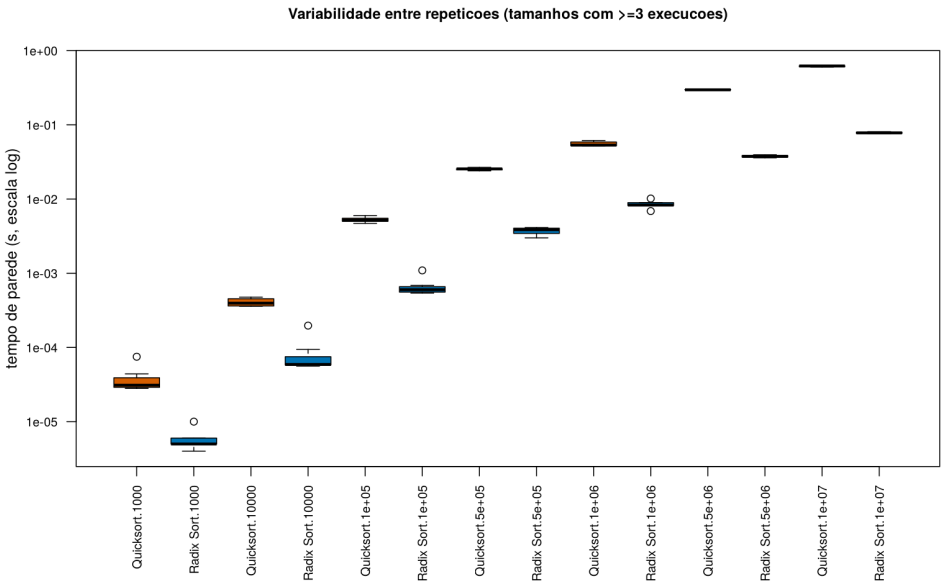


Figura 3: Variabilidade entre repetições (tamanhos com $\geq$3 execuções). Fonte: `resultados/graficos_r/analise.R`.

3.2 Análise Estatística

Regressão log-log ($\log(\textit{tempo}) = \log(a) + k\log(n)$), ajustada sobre as médias por tamanho, estima o expoente de crescimento empírico:

Tabela 5: Regressão log-log do tempo de execução (relógio de parede) em função de n.

Algoritmo	k estimado	IC 95%	R^2 ajustado
Quicksort	1,0555	[1,0459 ; 1,0651]	0,9999
Radix Sort	1,0227	[1,0018 ; 1,0436]	0,9993

Ambos os ajustes são excelentes ($R^2 > 0{,}999$). O expoente do **quicksort (1,0555) é visivelmente maior** que o do **radix sort (1,0227)**, refletindo o fator $\log n$ extra de $\Theta(n \log n)$ presente no quicksort e ausente no radix sort. O intervalo de confiança do radix sort **contém 1,0**, consistente com $\Theta(n)$ teórico; o do quicksort **não contém 1,0**, evidência de crescimento genuinamente superlinear.

Teste pareado (Wilcoxon signed-rank, repetições com a mesma entrada para os dois algoritmos como pares naturais, $n = 68$ pares, tamanhos de 10^3 a 10^8): $V = 2346$, $p \approx 7{,}8 \times 10^{-13}$. O quicksort foi mais lento que o radix sort em **100% dos 68 pares** comparados — diferença estatisticamente altamente significativa e sistemática, não um artefato de média.

3.3 Tamanhos extremos (planilha oficial, até 2 bilhões)

Tabela 6: Tempos medidos com `clock()`, execução única, nos 14 tamanhos oficiais da planilha do professor. Fonte: `resultados/tempos_planilha_clock.csv`.

Tamanho	Quicksort (s)	Radix Sort (s)
$T(10^6)$	0,057	0,007
$T(2 \times 10^6)$	0,111	0,015
$T(5 \times 10^6)$	0,334	0,038
$T(10^7)$	0,618	0,079
$T(2 \times 10^7)$	1,286	0,156
$T(5 \times 10^7)$	3,415	0,414
$T(10^8)$	7,141	0,849
$T(2 \times 10^8)$	16,154	1,518
$T(5 \times 10^8)$	38,099	4,012
$T(10^9)$	81,306	12,322
$\mathbf{T(2 \times 10^9)}$	**222,180**	**503,456**
$T(6 \times 10^9)$	não testado	não testado
$T(8 \times 10^9)$	não testado	não testado
$T(10^{10})$	não testado	não testado

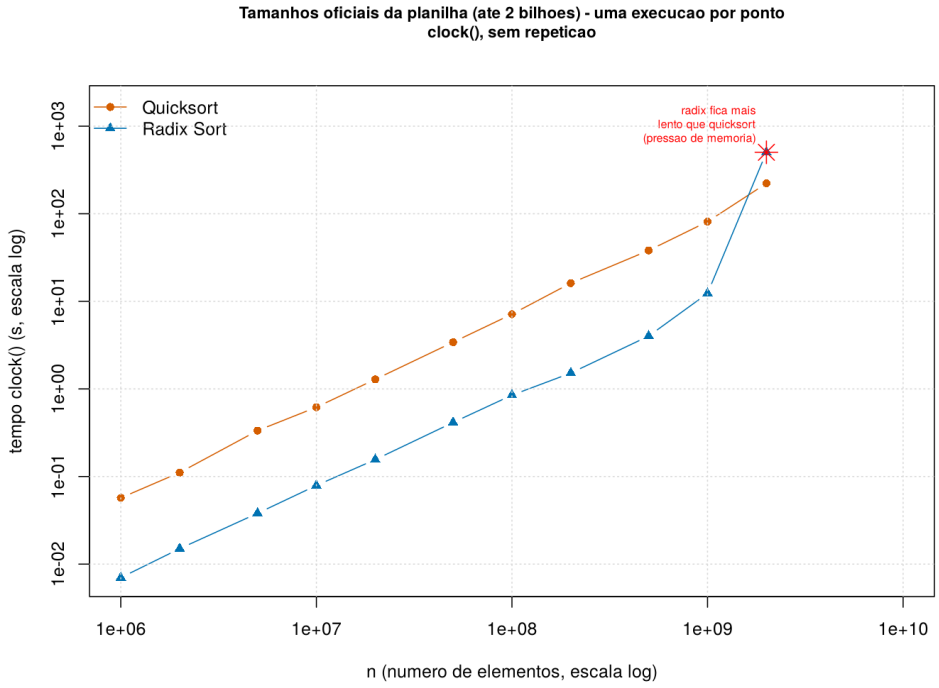


Figura 4: Tamanhos oficiais da planilha, até 2 bilhões de elementos. Fonte: `resultados/graficos_r/analise.R`, dados em `resultados/tempos_planilha_clock.csv`.

O resultado em $T(2 \times 10^9)$ inverte a tendência: pela primeira vez o radix sort (503,5 s) é **mais lento** que o quicksort (222,2 s). Não é erro de medição — é consequência direta da arquitetura de memória:

- Em $n = 2 \times 10^9$, um array de `int` (4 bytes) ocupa **8 GB**.
- O **quicksort é in-place**: usa só esses 8 GB.
- O **radix sort aloca um array auxiliar do mesmo tamanho** (Seção 2.2): em pico, precisa de **∼16 GB** — praticamente toda a RAM física da máquina (16,0 GB, Seção 2.1).
- Isso força paginação em disco, ordens de grandeza mais lenta que acesso à RAM. O próprio ambiente registrado (Seção 2.1) mostra **28,8 GB de memória virtual em uso** num sistema com apenas 16 GB de RAM física — evidência direta de pressão de memória nesse momento.
- **Ressalva metodológica:** este ponto é uma única execução, sem repetição (diferente da Seção 3, que tem múltiplas repetições e desvio-padrão). A inversão é consistente com o raciocínio teórico acima e com a leitura de memória virtual registrada, mas não se pode descartar 100% um efeito de ruído do sistema sem repetir a medição — não feita para evitar o alto custo de tempo (cada execução de $T(2 \times 10^9)$ já leva vários minutos).

Este é um exemplo concreto de por que a **complexidade espacial importa na prática, não só na teoria**: o radix sort tem melhor complexidade de tempo assintótico ($\Theta(n)$ vs. $\Theta(n \log n)$), mas sua maior exigência de memória ($O(n + b)$ com constante efetiva 2, pelo array duplicado) o torna mais lento assim que o tamanho do problema se aproxima do limite de RAM física disponível.

3.4 Mesma planilha, medida com `gettimeofday()`

A tabela da Seção 3.3 usa `clock()`, que mede **tempo de CPU** (apenas os ciclos em que o processo esteve de fato executando instruções). Os mesmos 14 tamanhos oficiais foram remedidos com `gettimeofday()`, que mede **tempo de relógio de parede** (*wall-clock*): o intervalo real decorrido, incluindo tempo em que o processo ficou bloqueado — por exemplo, aguardando o sistema operacional satisfazer uma falha de página durante paginação em disco. Execução única por tamanho, como na medição anterior.

Tabela 7: Tempos medidos com `gettimeofday()` (tempo de relógio de parede), execução única, nos mesmos 14 tamanhos oficiais da planilha do professor. Fonte: `tempos/planilha/gettimeofday.csv`.

Tamanho	Quicksort (s)	Radix Sort (s)
$T(10^6)$	0,060103	0,009102
$T(2 \times 10^6)$	0,140042	0,016028
$T(5 \times 10^6)$	0,306008	0,045090
$T(10^7)$	0,672141	0,085645
$T(2 \times 10^7)$	1,330477	0,227832
$T(5 \times 10^7)$	4,320474	0,656880
$T(10^8)$	8,720561	1,087448
$T(2 \times 10^8)$	15,489256	1,646543
$T(5 \times 10^8)$	37,005234	9,020103
$T(10^9)$	110,744745	16,734537
$T(2 \times 10^9)$	**243,714830**	**632,344466**
$T(6 \times 10^9)$	não testado	não testado
$T(8 \times 10^9)$	não testado	não testado
$T(10^{10})$	não testado	não testado

A inversão em $T(2 \times 10^9)$ se mantém — e é ainda mais acentuada: sob relógio de parede, o radix sort (632,3 s) é **2,85× mais lento** que o quicksort (243,7 s), contra **2,27**× na medição de tempo de CPU (503,5 s vs. 222,2 s). Isso **reforça**, em vez de contradizer, a explicação por pressão de memória da Seção anterior: o tempo gasto em troca de páginas com o disco conta como tempo de relógio de parede decorrido, mas conta pouco ou nada como tempo de CPU (o processo fica *bloqueado*, não executando instruções, enquanto o SO resolve a falta de página) — logo, é esperado que `gettimeofday()` exponha esse custo de forma mais completa que `clock()` justamente no ponto em que a paginação em disco é mais severa.

De forma mais geral, em todos os 11 tamanhos testados o tempo de relógio de parede é igual ou maior que o tempo de CPU correspondente (como deveria ser, já que nenhum processo pode gastar mais CPU do que tempo decorrido), exceto por três pontos do quicksort ($T(5 \times 10^6)$, $T(2 \times 10^8)$ e $T(5 \times 10^8)$) em que o `gettimeofday()` registrou um valor até 8% **menor** que o `clock()` — fisicamente só possível como ruído de medição, consistente com a mesma ressalva já feita na seção anterior: ambas as medições são de execução única, sem repetição, e portanto sujeitas a variação do sistema (outros processos, *scheduling* do SO) que não pode ser separada de um efeito real sem repetir a medição. O caso mais chamativo é o radix sort em $T(5 \times 10^8)$: 4,012 s por `clock()` contra 9,020 s por `gettimeofday()` (mais que o dobro) — nesse ponto o array auxiliar do radix sort já ocupa ∼4 GB (2 GB de entrada + 2 GB de cópia), o que começa a pressionar a memória bem antes do colapso completo em $T(2 \times 10^9)$, mesmo sem ainda inverter a ordem dos dois algoritmos.

3.5 Tamanhos não testados ($T(6 \times 10^9)$, $T(8 \times 10^9)$, $T(10^{10})$)

Não testados por **dois limites independentes e cumulativos**:

1. **RAM física**: 6, 8 e 10 bilhões de `int`s exigiriam ∼24, 32 e 40 GB, respectivamente — mais que os 16 GB disponíveis, mesmo antes do array auxiliar do radix sort.
2. **Modelo de dados do Windows (LLP64)**: o campo `long int length` do `struct Array` (fornecido pelo professor) tem **64 bits no Linux** (modelo LP64, usado nos exemplos originais do professor), mas apenas **32 bits no Windows** (modelo LLP64, adotado pelo MinGW-w64 por seguir a ABI do Windows) — mesmo em compilação de 64 bits. Isso limita o tamanho máximo representável a $2^{31} - 1 \approx 2{,}147$ bilhões, independentemente da RAM disponível. É uma diferença de portabilidade real entre Linux e Windows, documentada aqui, não uma limitação do algoritmo.

Essas células foram marcadas "Sem memória"por uma verificação prévia no código (`benchmark_clock.c`), que evita a tentativa de alocação, já que seria inútil (RAM) e, para $n > 2^{31} - 1$, produziria comportamento indefinido pelo *overflow* do campo `length`.

3.6 Conclusão

Os dados confirmam a teoria (Seção 2) em praticamente toda a faixa testada: o radix sort é consistentemente de 5 a 9× mais rápido que o quicksort, com vantagem relativa que **cresce** com n (de ∼6,6× em $n = 10^3$ a ∼9,1× em $n = 2 \times 10^8$) — compatível com o fator $\log n$ extra do quicksort, que pesa cada vez mais conforme n cresce. A regressão log-log confirma estatisticamente a diferença de expoentes e o teste pareado confirma que a vantagem do radix sort é sistemática ($p < 10^{-12}$). Porém, em $n = 2 \times 10^9$, essa vantagem se inverte na prática por causa do maior custo de memória do radix sort colidir com o limite físico de RAM da máquina — um lembrete de que complexidade assintótica de tempo e desempenho real divergem quando a complexidade **espacial** de um algoritmo o aproxima dos limites físicos do hardware disponível.

4 Questões do Questionário do EP2

4.1 Questão 1 — Chave secreta

Método idêntico ao de `calcSecret.c`: inicializa o gerador congruente linear (constantes de `java.util.Random`: multiplicador `0x5DEECE66D`, incremento `0xB`, módulo 2^{48}) com `setSeed(semente)` e aplica `next(31)` repetidamente; o último valor retornado é a chave.

- Semente: RA do autor = `18245598`
- Iterações: `31`
- **Chave secreta:** `1906028835`

Valor puramente aritmético (independe de hardware/SO/compilador), reconfirmado executando `secretkey_gen.exe 18245598 31` no mesmo ambiente Windows usado para os benchmarks.

4.2 Questão 2 — Crítica ao método de geração da chave

O gerador (LCG — *Linear Congruential Generator*) é o mesmo de `java.util.Random`: $seed = (seed \cdot \texttt{0x5DEECE66D} + \texttt{0xB}) \bmod 2^{48}$, $next(bits) = seed \gg (48 - bits)$.

Pontos fortes para este uso: determinístico e reprodutível (mesma semente ⇒ mesma chave, sem precisar armazenar tabela); rápido; para fins de *identificação* (não de segurança), a qualidade estatística do LCG é irrelevante.

Fraquezas (crítica técnica, mesmo não comprometendo este uso):

- LCGs têm **estrutura em reticulado** (Marsaglia, 1968, "Random numbers fall mainly in the planes"): k-uplas consecutivas caem num número reduzido de hiperplanos paralelos, nunca preenchendo o espaço uniformemente.
- Os **bits baixos de um LCG têm período curto** — por isso `next(bits)` usa só os bits altos do estado. O uso de `next(31)` (31 bits mais significativos) é a escolha correta sob esse aspecto.
- Dado o módulo 2^{48} e 31 iterações fixas, há mais valores de saída possíveis (até $\sim 2 \times 10^9$) do que RAs plausíveis, tornando colisões entre dois RAs diferentes improváveis mas não impossíveis (paradoxo do aniversário) em turmas de dezenas a poucas centenas de alunos.
- Um LCG **não é adequado para chaves criptográficas** (previsível a partir de poucas saídas consecutivas) — irrelevante aqui, que é uso de identificação pública, não de sigilo.

4.3 Questão 3 — Código-fonte

Repositório oficial do projeto: https://github.com/igorfcfs/IAA-EP2-ComparacaoAlgoritmos — todo o código-fonte, scripts de benchmark, dados brutos e este próprio relatório estão disponíveis lá para análise detalhada pelo professor, caso deseje.

Em anexo a esta questão: `testSort.c`, reunindo em um único arquivo as duas implementações próprias de ordenação pedidas no EP2 (quicksort e radix sort), no mesmo padrão de uso de `start()/stop()/printCSVElapsed()` do `testSort.c` de exemplo fornecido pelo professor. As implementações são idênticas às de `quicksort.c` e `radixsort.c` do repositório (Seção 2), apenas consolidadas num só arquivo para facilitar a leitura.

Compilação (os três arquivos abaixo são os fornecidos pelo professor, sem nenhuma alteração):

```
gcc -O2 rng.c utilarray.c elapsedRunningTime.c testSort.c -o testSort
```

Execução: `./testSort [n]` — `n` é opcional (padrão 1 000 000). O programa gera um único array aleatório de tamanho n, faz uma cópia idêntica dele (mesmo controle experimental da Seção 2.4: os dois algoritmos ordenam exatamente a mesma entrada), ordena uma cópia com quicksort e a outra com radix sort, mede o tempo de cada ordenação com `clock()` e `gettimeofday()` e confere se cada resultado ficou corretamente ordenado, imprimindo uma linha CSV por algoritmo (`algoritmo;n;tempo_clock_s;tempo_wall_s;ordenado`). Testado neste exato formato para $n = 10^3$, 5×10^6 e o padrão 10^6, com `ordenado=1` em todos os casos.

Como cada coluna das tabelas de resultados foi gerada (dois mecanismos distintos, não um só): para a tabela de tempo médio com repetições (Seção 3, n de 1.000 a 200.000.000), o `main()` de `run_sort.c` **não** foi ajustado para rodar tudo internamente — cada execução roda um único teste (um algoritmo, um tamanho, uma repetição) e imprime uma linha CSV; um script externo (`bench.ps1`) orquestra o laço sobre tamanhos, repetições e algoritmo, chamando `gen_input.exe` (mesma entrada para os dois algoritmos) e depois `run_sort.exe` a cada iteração. Já a tabela oficial da planilha (Seção 3.3, 14 tamanhos até $T(10^{10})$, 1 execução cada) foi gerada por `benchmark_clock.c`, que tem um laço **interno** no próprio `main()`, percorrendo os 14 tamanhos em sequência numa única execução do programa.

O restante do código-fonte completo (não apenas os algoritmos de ordenação) está em `src/` no repositório acima: `rng.c/h`, `utilarray.c/h`, `elapsedRunningTime.c/h` (fornecidos pelo professor, sem alterações); `quicksort.c/h`, `radixsort.c/h` (implementação própria, Seção 2); `gen_`

`input.c`, `run_sort.c`, `benchmark_clock.c`, `secretkey_gen.c` (programas de suporte, próprios). Compilação de cada um na Seção 3.2.

4.4 Questão 4 — Preenchimento da planilha

Os dados necessários para o preenchimento — identificador (`1906028835`), informações de ambiente (Seção 2.1) e os 14 tempos oficiais por algoritmo (Tabela em Seção 3.3, disponíveis também em `resultados/tempos_planilha_clock.csv`) — estão prontos e documentados neste Relatório.

4.5 Questão 5 — Discussão científica

Ver Seção 3 (Resultados Experimentais e Discussão Científica), que cobre metodologia, resultados tabulados, análise estatística (regressão log-log e teste pareado de Wilcoxon), o achado do ponto de inversão em $T(2 \times 10^9)$ e sua explicação em termos de complexidade espacial, e os limites de representação em `long int` no Windows.

5 Fontes e Referências

- F. Nakano. *Enunciado e código-base do EP2, IAA*. https://github.com/FNakano/IAA-Apostila/tree/main/EP2.
- F. Nakano. *getPlatformInfo*. https://github.com/FNakano/IAA-Apostila/tree/main/getPlatformInfo.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3ª ed. — análise do quicksort por variáveis indicadoras (cap. 7) e do radix sort (cap. 8).
- G. Marsaglia. "Random numbers fall mainly in the planes". *Proceedings of the National Academy of Sciences*, 61(1), 1968 — estrutura em reticulado de LCGs.
- Oracle. Documentação de `java.util.Random` — constantes do gerador congruente linear usado em `rng.c`.
- R Core Team. *R: A Language and Environment for Statistical Computing*, versão 4.3.3 — usado em `resultados/graficos_r/analise.R` para a regressão log-log, teste de Wilcoxon e os gráficos deste Relatório.